

Predicción de Popularidad de Canciones de Spotify

Miguel Gordillo
Universidad Simón Bolívar
Caracas, Venezuela
migueluchogordillo@gmail.com

Blanyer Vielma
Universidad Simón Bolívar
Caracas, Venezuela
vblanyer@gmail.com

Franco Murillo
Universidad Simón Bolívar
Caracas, Venezuela
murillo.franc@gmail.com

RESUMEN

El presente proyecto tuvo como objetivo predecir la popularidad de canciones en Spotify utilizando técnicas de aprendizaje de máquina, en particular una red neuronal, usando la librería PyTorch del lenguaje de programación Python.

Inicialmente se planteó como un problema de regresión para estimar un valor numérico entre 0 y 100, pero debido a la baja correlación de los atributos de las canciones del dataset con respecto a la popularidad, se reformuló como una tarea de clasificación binaria (popular o no) dependiendo si el valor de la popularidad de la canción es mayor o menor a 50.

Se entrenó una red neuronal, donde fueron probadas diferentes configuraciones en diversas pruebas, optando finalmente por una red de dos capas, con 64 neuronas en la primera capa oculta y 32 en la segunda, utilizando BCEWithLogitsLoss como función de pérdida, LeakyReLU como función de activación entre las capas ocultas, Sigmoide para la capa output y ajustando el umbral de decisión a 0.7 para optimizar métricas como precisión y recall.

El modelo alcanzó un recall de hasta 50,7%, una precisión de 40,4% y un accuracy del 79,1%, lo que indica una alta aleatoriedad por parte del modelo para predecir si una canción será popular o no, lo que evidencia falta de correlación de los atributos con la popularidad.

Se concluye que, con mejoras en la calidad del dataset y el uso de arquitecturas más complejas, es posible aumentar la precisión del modelo y su aplicación en entornos reales de curaduría musical.

I. INTRODUCTION

En la era del streaming digital, plataformas como Spotify han transformado la manera en que se consume música, generando enormes volúmenes de datos sobre hábitos de escucha, características sonoras y métricas de popularidad. Esta riqueza de información abre la puerta a nuevas oportunidades en el análisis predictivo, donde la inteligencia artificial puede ayudar a anticipar el éxito de una canción antes de su lanzamiento o en sus primeras etapas de difusión. Este proyecto se enfoca en el desarrollo y entrenamiento de una red neuronal para predecir la popularidad de canciones en Spotify, utilizando como dataset datos públicos sobre atributos musicales (como energía, valencia, tempo, duración, etc.) obtenidos de la página web kaggle, especializada para buscar conjuntos de datos. La elección de redes neuronales se fundamenta en su capacidad para modelar relaciones no lineales

y complejas entre variables, lo cual es especialmente útil en contextos donde el éxito musical depende de múltiples factores interrelacionados. Inicialmente, el objetivo del proyecto era construir un modelo capaz de predecir la popularidad de una canción en Spotify como un valor numérico entre 0 y 100 usando la librería PyTorch una librería del lenguaje de programación Python especializada en aprendizaje profundo, junto con Pandas y NumPy para el procesamiento y manejo de los datos. Sin embargo, durante la fase de exploración y desarrollo, se identificaron limitaciones importantes en la calidad, distribución y consistencia de los datos disponibles, lo que dificulta obtener predicciones precisas en escala continua. Como resultado, se redefinió el problema como una tarea de clasificación binaria: determinar si una canción puede considerarse "popular" o "no popular" según un umbral definido. Este cambio permitió simplificar el modelo, mejorar la interpretabilidad de los resultados y facilitar la evaluación mediante métricas como precisión y recall. No obstante, como se analizará más adelante, la baja correlación entre los atributos disponibles y la popularidad dificultó obtener resultados predictivos satisfactorios.

II. DESCRIPCIÓN DE LA SOLUCIÓN

Para abordar el problema de predicción de la popularidad, se emplea una red neuronal supervisada entrenada con backpropagation. La técnica de aprendizaje utilizada será el aprendizaje supervisado, ya que se cuenta con ejemplos etiquetados de canciones junto con su nivel de popularidad. No se desarrollará la arquitectura desde cero, sino que se utilizarán herramientas y librerías, en particular, se empleará PyTorch, una librería del lenguaje de programación Python ampliamente utilizada en el desarrollo de modelos de aprendizaje profundo, que permite definir redes neuronales de forma flexible y eficiente, NumPy para el manejo de arreglos y cálculos sobre estos, y Pandas para manejar archivos. Inicialmente se utilizará como función de pérdida MSE para clasificación binaria, y el optimizador Adam para el ajuste de pesos. La arquitectura del modelo será ajustada experimentalmente, considerando el número de capas ocultas, neuronas por capa, funciones de activación, tasa de aprendizaje y número de iteraciones, con el objetivo de minimizar el RMSE.

Descripción del Dataset

El modelo será entrenado utilizando un conjunto de datos extraído de kaggle, que incluye atributos musicales como

nombre del artista, energía, valencia, tempo, duración, instrumentalidad, entre otros. Cada instancia representa una canción, y está acompañada por una etiqueta que indica la popularidad de la canción con un número de 0 a 100.

Contenido del Dataset

El conjunto de datos fue extraído por un tercero en Kaggle mediante la API de Spotify, y posteriormente fue actualizado para el objetivo del estudio. Contiene varios atributos importantes que almacena la plataforma para el análisis de las pistas musicales.

Descripción de los Atributos del Dataset

A continuación, se presentan las columnas de la tabla del dataset:

- **track_id**: Id único de la pista musical
- **artists**: Nombre de los artistas que participan en la pista musical.
- **album_name**: Nombre del álbum que incluye la pista musical.
- **track_name**: Nombre de la pista musical.
- **popularity**: Número entre el rango 1 y 100 que representa la popularidad de la pista musical.
- **duration_ms**: Tiempo de duración de la pista musical en milisegundos.
- **explicit**: Valor booleano que indica si la pista musical contiene contenido explícito en su letra.
- **danceability**: Valor entre 0.0 y 1.0 que describe la idoneidad de una pista para bailar según una combinación de elementos musicales, como el tempo, la estabilidad rítmica, la intensidad del compás y la regularidad general.
- **energy**: La energía es una medida de 0.0 a 1.0, representa una medida perceptual de intensidad y actividad (Normalmente, las pistas energéticas se perciben rápidas, fuertes y ruidosas.).
- **key**: Representa la tonalidad de la pista como números enteros incluyendo al cero representando la notación estándar musical. Si no se determina una tonalidad el número representativo es -1.
- **loudness**: Decibelios de la pista.
- **mode**: Representa la modalidad mayor o menor musical de la cual deriva la pista musical, representado como 1 y 0, respectivamente.
- **speechiness**: La fonética detecta la presencia de palabras habladas en una pista. Cuanto más se parezca la grabación a una voz, más cercano a 1.0 será el valor del atributo. Los valores superiores a 0.66 describen pistas que probablemente estén compuestas completamente de palabras habladas. Los valores entre 0.33 y 0.66 describen pistas que pueden contener tanto música como voz, ya sea en secciones o en capas, incluyendo casos como el rap. Los valores inferiores a 0.33 probablemente representan música y otras pistas que no son de voz.
- **acousticness**: Indicador de confianza creciente que representa si la pista es acústica.

- **instrumentalness**: Indicador de confianza respecto al nivel de instrumentalidad que posee la música en contraste con voces.
- **liveness**: Indicador de presencia de voces ajenas a los artistas.
- **valence**: Indicador creciente que representa el nivel de emociones negativas y positivas de la pista musical.
- **tempo**: BPM por segundo de una pista musical.
- **time_signature**: Número entero que representa la notación estándar del compás.
- **track_genre**: Género al cual la pista musical pertenece.

Descripción del pre-procesamiento de los datos

Primero se limpió la data para que cada fila pudiera ser procesada correctamente. Para ello, se usa el archivo `utils/clean_lines.py` el cual procesa todo el conjunto de datos del `.csv` para eliminar caracteres no deseados y eliminar filas consideradas como problemáticas, por ejemplo, filas que contenían títulos y nombres de artistas muy difíciles de limpiar con expresiones regulares. Este nuevo conjunto se guarda como una copia del conjunto original pero limpiado.

Esta función solo se ejecutó una vez fuera de la ejecución principal del programa, ya que solo es necesario hacerlo una vez.

Luego, mediante el archivo `data/preprocessor.py` se hace la separación de atributos, identificando cuál será el target y cuales son irrelevantes, como el `"track_id"`. Ahora, dado que una red neuronal solo puede trabajar con datos numéricos, se hizo una codificación para los atributos de tipo texto, como el nombre del artista, de tal forma que cada valor distinto, tuviera una codificación numérica única. De igual forma, el atributo `"duration_ms"` es dividido entre 1000 para tener su escala en segundos.

Al final de esta etapa, se separa directamente el conjunto de entrenamiento del conjunto de prueba, usando un randomizador por semilla para garantizar que cada vez que se ejecute el programa se obtenga una división diferente, pero que se mantenga esta división a lo largo de toda la ejecución del mismo. De igual forma, se preparan los tensores usando los atributos y el target de ambos conjuntos para que puedan ser procesados por la red neuronal, ya que esto es lo que usa PyTorch para realizar el entrenamiento y la prueba.

A pesar de que existe una mejora al momento de entrenar el modelo, ya que hay una diferencia en entrenarlo con pocas épocas (primeras filas de la tabla) que con muchas, estos resultados llaman la atención, dado que diferentes hiperparámetros no resultaron en una mejora concreta del modelo. Esto no pareció a primera vista normal, por lo que inicialmente se pensó que la red tenía algún error en la construcción, pero tras un análisis posterior, esta conclusión no fue correcta. Por lo que se decidió revisar la correlación de los atributos con respecto a la popularidad, algo no realizado pero muy importante en la fase de preprocesamiento y análisis del conjunto de datos. Al evaluar la correlación de cada atributo con respecto a la popularidad, y combinaciones de estos, se obtuvieron los siguientes resultados:

Diseño de la solución

La organización del modelo se divide en diferentes carpetas, cada una conteniendo los archivos necesarios para ejecutar, probar el modelo, permitiendo de esta forma una mayor modularidad y facilidad de aprendizaje del código, para un mejor entendimiento de cómo funciona el mismo. La estructura es la siguiente:

- **Data:** Contiene el conjunto de datos junto con el archivo principal para preprocesar la data.
- **Models:** Contiene la estructura de la red neuronal, esto es, las capas intermedias, número de neuronas, y funciones de activación usadas.
- **Saved_models:** Contiene la red neuronal ya entrenada, esto es, los pesos obtenidos del entrenamiento para cada neurona.
- **Train:** Contiene la red neuronal ya entrenada, esto es, los pesos obtenidos del entrenamiento para cada neurona.
- **Utils:** Contiene diferentes archivos auxiliares para limpiar la data, obtener correlaciones entre los atributos y la popularidad, configuraciones globales como el número de neuronas o la tasa de aprendizaje, probar el modelo y realizar gráficas.

El pipeline de la ejecución comienza con la función `main`, la cual se encarga de preparar la data a procesar (limpieza, normalización, etc), configurar la red neuronal, configurar los parámetros de entrenamiento y realizar el entrenamiento con el conjunto de entrenamiento, guardar el modelo y probar el modelo con el conjunto de prueba.

Ahora, en cuanto a la arquitectura inicial, se construyó la red neuronal usando las facilidades de PyTorch y las configuraciones generales. Inicialmente se empezó con una red neuronal de dos capas intermedias y una de salida (3 capas), y como función de activación LeakyReLU entre cada capa, debido a que el atributo `loudness` contiene valores negativos, y esta función de activación permite que cierta información pueda pasar a la siguiente neurona, sin generar el problema de neuronas muertas.

Luego, para realizar el entrenamiento de la red usando PyTorch, es necesario definir cuáles serán la función de pérdida, el optimizador (el algoritmo encargado de ajustar los pesos), las épocas y el batch size, así como el modelo base (la red neuronal sin entrenar) y los atributos junto con el target de cada dato del conjunto de entrenamiento.

En principio se usó el **MSE** (Mean Squared Error) y como optimizador Adam (ver bibliografía). Las épocas se variaron conforme se fue experimentando, y el batch size se dejó en 32. Cabe mencionar que el batch size define cuántas muestras del conjunto de datos se procesan juntas antes de actualizar los pesos del modelo, ya que entrenar con todo el dataset a la vez (lo que se llama entrenamiento en batch completo) puede ser muy lento y consumir mucha memoria, por lo que realizar esto es útil dado que el dataset contiene unos 110000 datos.

Por otro lado, se usaron los **tensores** de PyTorch, los cuales son la estructura de datos fundamental para representar información numérica. Son similares a los arrays de NumPy, pero

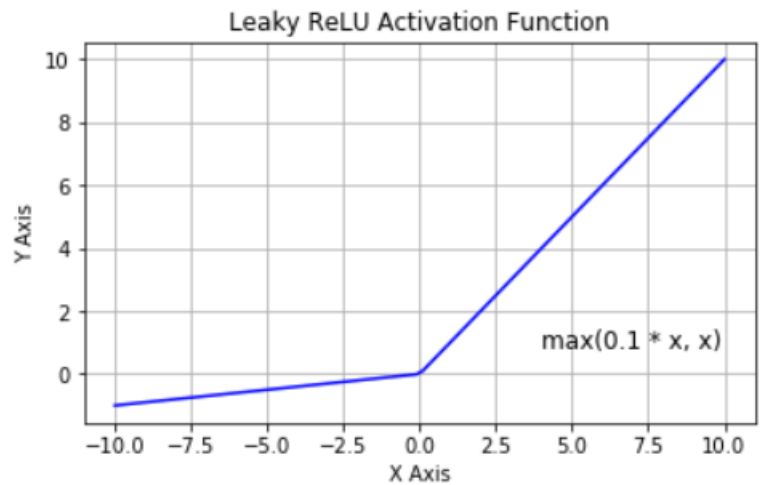


Fig. 1. Función de Activación LeakyReLU

con capacidades adicionales para operar en GPU y construir redes neuronales. Un tensor es una matriz multidimensional que puede tener desde cero dimensiones (un escalar) hasta muchas dimensiones (como una imagen 3D o una secuencia temporal). Son la base de todos los cálculos: entradas, pesos, salidas, gradientes; y permiten operaciones matemáticas eficientes como suma, multiplicación, convoluciones, etc, además de que se pueden mover fácilmente entre CPU y GPU para acelerar el entrenamiento.

III. EXPERIMENTOS Y RESULTADOS

Descripción del Entrenamiento

Como se detalló anteriormente, toda la estructura, tanto para realizar el entrenamiento, como para probar el modelo, se realiza en la fase de preprocesamiento, donde se toma un 80% de los datos para entrenar y un 20% para probar (en la mayoría de las pruebas), de forma aleatoria. Además selecciona la función de pérdida y el optimizador justo antes de entrenar el modelo. En cuanto a los hiper parámetros, estos se definen en un archivo exclusivo (`utils/config`) que permite realizar de forma rápida diferentes cambios en cada uno de estos para realizar diferentes pruebas. Allí se definen el número de neuronas de la primera capa (donde las demás capas tendrán en general la mitad de neuronas de la capa anterior), el número de épocas, el tamaño del batch y la tasa de aprendizaje. Luego, durante la fase de entrenamiento, se realiza la separación en batches del conjunto, y se realiza el backpropagation junto con la actualización de pesos, por cada batch durante el número definido de épocas. Una vez finalizado el entrenamiento, el modelo se guarda para poder ser probado. En este punto, se realizaron diferentes pruebas para probar el rendimiento del modelo, observando el cambio del RMSE, donde un menor valor de este indicaba un menor error en la predicción del valor real de la popularidad.

La convergencia de la función de pérdida (el RMSE), se puede observar en la siguiente gráfica:

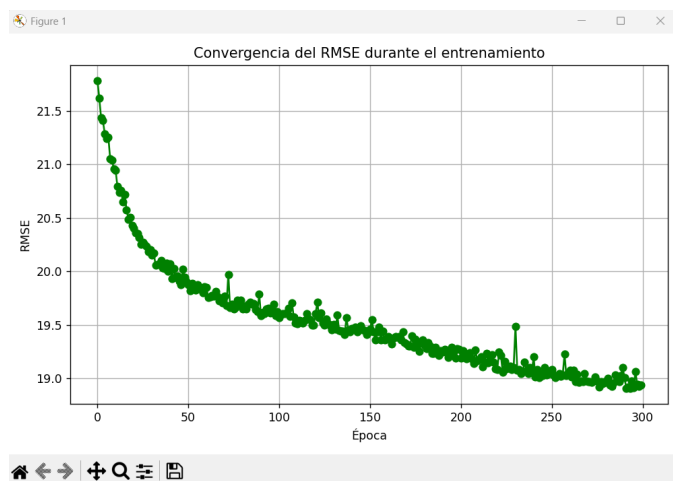


Fig. 2. Convergencia RMSE

De la gráfica se puede observar que el error disminuye durante el entrenamiento, con ciertos puntos donde pareciera haber un overfitting, pero en general con una curva que disminuye con el tiempo.

Por otra parte, el siguiente gráfico muestra las predicciones hechas por el modelo en comparación con los valores reales del Dataset:

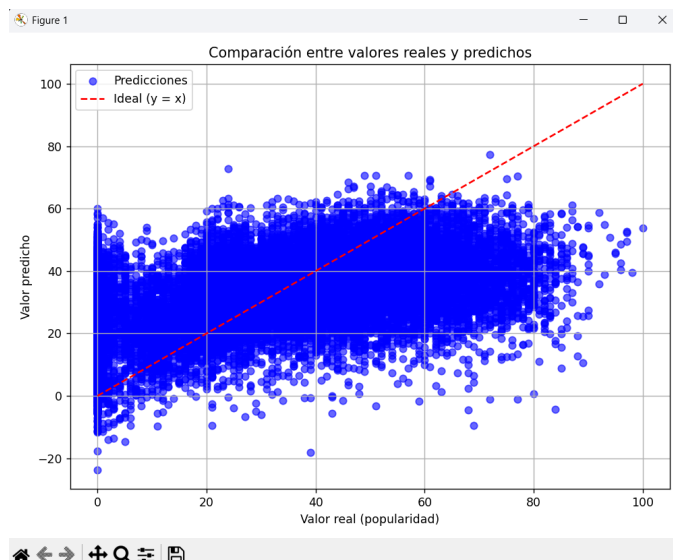


Fig. 3. Convergencia RMSE

En esta gráfica se puede observar que no existe una tendencia general al momento de predecir correctamente la popularidad. El modelo tiende a sobreestimar las popularidades bajas y sobreestimar las altas.

Estas dos últimas gráficas se mantuvieron similares para todas pruebas realizadas.

Por último, las pruebas realizadas, junto a las métricas y sus resultados se pueden observar en la siguiente figura:

Capas	Distribución Neuronas	Batch Size	Learning Rate	Épocas	Datos de Entrenamiento	MSE en Test	RMSE en streams reales
1	64 -32	32	0.001	2	80%	479.1087	22
1	64 -32	32	0.001	10	80%	440.7268	21
2	64 -32	32	0.001	300	80%	389.0995	20
2	64 -32	32	0.0001	300	80%	420.0816	20
2	64 -32	32	0.001	300	70%	419.2123	20
3	64 -32 -16	32	0.001	300	80%	375.6372	19
3	64 -32 -16	32	0.0005	600	80%	381.5968	20
3	64 -32 -16	32	0.0005	300	80%	383.0613	20
4	64 -64 -32 -16	32	0.001	300	80%	381.3098	20
4	64 -32 -16 -8	32	0.001	300	80%	377.5063	19
4	128 -64 -32 -16	32	0.001	300	80%	381.2831	20
5	64 -32 -16 -8 -4	32	0.001	300	80%	392.6750	20
5	128 -64 -32 -16 -8	32	0.001	300	80%	389.6880	20

Fig. 4. Tabla Comparativa de Pruebas

Se puede observar que para la mayoría de las pruebas, se obtuvo un RMSE de 20, lo que significa que en promedio, para cada canción, el modelo tiene un error de predicción de 20 puntos. Esto quiere decir, por ejemplo, que si una canción tiene una popularidad de 50, el modelo puede generar una predicción que se encuentre entre 30 y 70, que aunque no es un error circunstancial, no es lo suficientemente preciso como para considerar usar el modelo con la suficiente confianza. Además, esto hace que el modelo prediga popularidades negativas, como se puede ver en el segundo gráfico, lo cual no puede ocurrir.

A pesar de que existe una mejora al momento de entrenar el modelo, ya que hay una diferencia en entrenarlo con pocas épocas (primeras filas de la tabla) que con muchas, estos resultados llamaron la atención, dado que diferentes hiperparámetros no resultaron en una mejora concreta del modelo. Esto no pareció normal, por lo que inicialmente se pensó que la red tenía algún error en su construcción, pero tras un análisis posterior, se concluyó que esa no era la razón. Por lo que se decidió revisar la correlación de los atributos respecto a la popularidad, algo no realizado antes, pero muy importante en la fase de preprocesamiento y análisis del conjunto de datos. Los resultados de la evaluación de la correlación de cada atributo con respecto a la popularidad, y combinaciones de éstos, se observan en las siguientes tablas:

TABLE I
CORRELACIONES INDIVIDUALES

loudness	0.044859
duration_ms	0.019203
track_genre	0.018514

TABLE II
INTERACCIONES DOBLES MÁS CORRELACIONADAS

Atributo 1	Atributo 2	Correlación
loudness	speechiness	0.077984
loudness	instrumentalness	0.072157
duration_ms	track_genre	0.041283

TABLE III
INTERACCIONES TRIPLES MÁS CORRELACIONADAS

Atributo 1	Atributo 2	Atributo 3	Correlación
energy	loudness	instrumentalness	0.105678
loudness	speechiness	instrumentalness	0.082815
loudness	instrumentalness	valence	0.082011

TABLE IV
INTERACCIONES CUÁDRUPLES MÁS CORRELACIONADAS

Atributo 1	Atributo 2	Atributo 3	Atributo 4	Correlación
energy	loudness	instrumentalness	tempo	0.107432
energy	loudness	speechiness	instrumentalness	0.106357
energy	loudness	instrumentalness	valence	0.094669

De las 4 tablas se puede observar que para los atributos únicos, el que tiene mayor correlación es loudness, y es apenas de 4%, y para las demás combinaciones este atributo sigue apareciendo, obteniendo como máximo una correlación de 10% para la combinación triple de loudness, energy y instrumentalness, y la combinación cuádruple de energy, loudness, instrumentalness y tempo.

Estos resultados muestran una fuerte evidencia de la poca relación (prácticamente nula) entre los atributos y la popularidad, lo que evidencia la poca precisión del modelo para predecir el número específico de la popularidad. En este punto se decidió realizar lo siguiente para intentar mejorar el aprendizaje de la red:

- Añadir el atributo "release_date" a cada canción usando directamente la API de Spotify, dado que en la descripción del conjunto se habla que las canciones populares tendrán una mayor popularidad si se publicaron recientemente, por lo que este atributo debería tener una correlación alta con la popularidad.
- Actualizar la popularidad de cada canción a la actual de la API de Spotify.
- Reformular el problema a uno de clasificación binaria, donde la red solo tenga predecir si una canción será popular (valor 1) o no (valor 0).

El umbral de popular será para aquella que tenga popularidad mayor o igual a 50, y no popular las que tengan menos, dado que hay más canciones no populares que populares, con lo que las dos clases estarán más balanceadas. También, se eliminaron los atributos con menos correlación (los más cercanos a cero) y atributos de texto, para evitar que la codificación de éstos perjudique el aprendizaje de la red, y para que la red pueda detectar patrones en los atributos más importantes. Los atributos eliminados fueron "danceability", "key", "acousticness", "liveness", "artists", "album_name", "track_name" y "time_signature". Todo esto se realiza en el preprocesamiento de los datos. Para actualizar la popularidad y agregar el atributo "release_date" se usó el script utils/add_data.py por separado, el cual se encarga de realizar todo el proceso de conectarse a la API, buscar cada canción por su track_id y aplicar la actualización de cada canción, de forma automática. Realizado esto, se volvió a revisar las correlaciones, para comprobar si el nuevo atributo "release_date" tiene una correlación más alta con la popularidad. Los resultados se observan en las siguientes tablas:

TABLE V
CORRELACIONES INDIVIDUALES

Atributo	Correlación
loudness	0.044859
release_date	0.024254
duration_ms	0.019203

TABLE VI
INTERACCIONES DOBLES MÁS CORRELACIONADAS

Atributo 1	Atributo 2	Correlación
loudness	speechiness	0.077984
loudness	instrumentalness	0.072157
loudness	release_date	0.043095

TABLE VII
INTERACCIONES CUÁDRUPLES MÁS CORRELACIONADAS

Atributo 1	Atributo 2	Atributo 3	Atributo 4	Correlación
energy	loudness	instrumentalness	tempo	0.107432
energy	loudness	speechiness	instrumentalness	0.106357
energy	loudness	instrumentalness	release_date	0.105791

De las 3 tablas se puede observar que, aunque ahora el atributo "release_date" aparece entre los primeros en tener mayor correlación, sigue siendo muy baja, incluso por debajo del atributo loudness. Esto quiere decir que esta aproximación no resuelve el problema de la baja correlación, pero si facilita el entrenamiento del modelo. Dado que ahora el problema es de clasificación binaria, se cambió la función de activación a sigmoide y como función de pérdida a BCEWithLogitsLoss (Binary Cross Entropy), la cual es una función de pérdida en PyTorch diseñada específicamente para tareas de clasificación binaria, y combina dos pasos en uno solo: la activación sigmoide y la pérdida de entropía cruzada binaria. También se usan las métricas de accuracy, precision y recall, útiles para este tipo de problemas. Accuracy mide la proporción de predicciones correctas sobre el total de predicciones:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Precision mide de todas las veces que el modelo predijo "positivo", ¿cuántas veces acertó?. Esto es útil cuando el costo de un falso positivo es alto, en nuestro caso, equivale a no recomendar canciones no populares:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Recall mide de todos los casos realmente positivos, ¿cuántos detectó el modelo?. Esto es útil cuando el costo de un falso negativo es alto. En nuestro caso equivale a no perder ninguna canción potencialmente popular:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Nuevamente, se realizó el entrenamiento con la red neuronal y el conjunto de datos actualizado. Tras probar diferentes configuraciones para la red, los resultados no variaron significativamente, excepto al cambiar el threshold al momento de que el modelo tome la decisión, considerando la posibilidad de asignar a la canción la popularidad. Este parámetro es parte de las métricas recall y precisión, y aumenta o disminuye el porcentaje correcto de canciones calificadas como populares. Los resultados se observan en la siguiente figura:

Capas	Distribución Neuronas	Batch Size	Learning Rate	Épocas	Datos de Entrenamiento	Threshold	Accuracy	Precision	Recall
2	128 - 64	32	0.001	200	80%	0.3	61,4%	28,1%	82,7%
2	128 - 64	32	0.001	200	80%	0.5	71,8%	33,5%	68%
2	128 - 64	32	0.001	200	80%	0.7	79,1%	40,4%	50,7%

Fig. 5. Resultados del nuevo entrenamiento

De la figura se puede observar que a medida que aumenta el threshold, aumentan accuracy y precision, pero disminuye recall. Esto quiere decir que aumenta el número general de predicciones, pero disminuye la detección de canciones populares, además de que se producen muchos falsos positivos. Dado que se desea maximizar la confianza del modelo para predecir un hit (una canción popular), se optó por el threshold de 0.7, ya que, aunque disminuye el número de veces que el modelo detecta una canción popular (recall), aumenta la confianza de la detección de las canciones que si detecta como populares (precision). Sin embargo, estos resultados aún no son lo suficientemente buenos como para confiar de manera plena y ciega en el modelo. Se sigue con la hipótesis de que la causa de este problema es principalmente la baja correlación de los atributos con respecto a la popularidad. Por lo tanto, por más que se mejore la estructura de la red, los resultados siguen estando más cerca de la aleatoriedad que de un resultado confiable y preciso, ya que la red intenta aprender patrones que prácticamente no existen en el conjunto de datos.

IV. CONCLUSIÓN Y RECOMENDACIONES

A pesar de que al refactorizar el problema a uno de clasificación se logró obtener un recall de 50,7%, un precision de 40,4%, y un accuracy del 79,1% (los mejores resultados del entrenamiento), se sigue observando una alta aleatoriedad al momento de predecir correctamente la popularidad de una canción, sobretodo si la canción sí es popular, lo que evidencia que el principal factor para que la red neuronal pueda aprender correctamente no es la configuración de ésta, sino la correlación de los atributos de cada canción con su popularidad. Con estos resultados, se recomienda, para implementaciones futuras:

- Una mejor elección del conjunto de datos y de prueba, que representen mejor las conexiones entre los atributos de una canción y su popularidad.
- Asignar nuevos atributos mediante la combinación de algunos existentes, por ejemplo energy más loudness, o crearlos por cuenta propia, como por ejemplo "presencia de voz femenina", que puedan producir una mejor correlación con la popularidad.
- Explorar estructuras más robustas y/o complejas como transformers, embeddings y mayor número de capas y neuronas en la red, en caso de contar con la capacidad computacional y el tiempo necesario, con el fin de detectar patrones débiles y, eventualmente, descubrir relaciones que no fueron captadas durante el desarrollo de este proyecto con el dataset utilizado.

Con las mejoras realizadas, este modelo puede resultar útil para apoyar decisiones de promoción musical que permitan a cantantes o empresas realizar una correcta inversión en sus canciones, con el fin de maximizar la popularidad de la misma con el mínimo riesgo.

REFERENCIAS

Maharshi Pandya (2023). Spotify Tracks Dataset. Hugging Face. Disponible en:
<https://huggingface.co/datasets/maharshipandya/spotify-tracks-dataset>

Optimizador de Adam. Disponible en:
<https://www.geeksforgeeks.org/deep-learning/adam-optimizer/>

Tensores PyTorch. Disponible en:
https://docs.pytorch.org/tutorials/beginner/introyt/tensors_deeper_tutorial.html

API de Spotify. Disponible en:
<https://developer.spotify.com/documentation/web-api>

Funciones de activación. Disponible en:
<https://developers.google.com/machine-learning/crash-course/neural-networks/activation-functions?hl=es-419>

Funciones de pérdida. Disponible en:
<https://www.ibm.com/es-es/think/topics/loss-function>